

Introduction to Automated Testing

Software Testing – Automated Testing Assignment

Student: Moshail Jafar

Project: Introduction to Automated Testing

GitHub: <https://github.com/Mishomoon/automated-testing>

1. Introduction

The purpose of this assignment was to create a simple Node.js project and implement automated testing with Jest.

The application contains a divide() function that divides two numbers and validates the input.

The project was tested using both positive and negative test cases.

The final project was also uploaded to GitHub.

2. Creating the Node.js Project

I started by creating the project folder:

automated-testing

The project contains the following main files and folders:

automated-testing

|

|— src

| |— calculator.js

| |— index.js

|

|— __tests__

| |— calculator.test.js

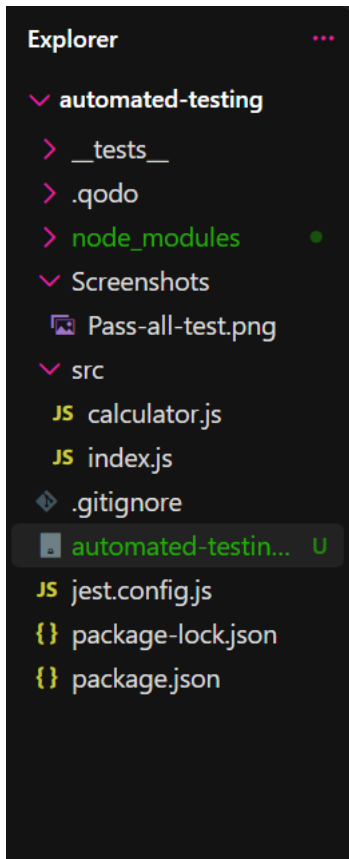
|

|— Screenshots

| |— Pass-all-test.png

|

```
├─ package.json
├─ package-lock.json
├─ jest.config.js
└─ .gitignore
```



3. Creating the divide() Function

I created the file:

src/calculator.js

The file contains the divide() function.

The function checks that:

- both arguments are numbers
- the arguments are not NaN
- the second number is not zero
- valid numbers are divided normally

The function also throws an appropriate error when invalid input is given.

```
1  export function divide(a, b) {
2      if (typeof a !== "number" || typeof b !== "number") {
3          throw new TypeError("Both arguments must be numbers");
4      }
5
6      if (Number.isNaN(a) || Number.isNaN(b)) {
7          throw new TypeError("Arguments cannot be NaN");
8      }
9
10     if (b === 0) {
11         throw new RangeError("Division by zero is not allowed");
12     }
13
14     return a / b;
15 }
```

4. Creating index.js

I created:

src/index.js

The purpose of this file is to run the application and check that the function works.

The application divides two numbers and prints the result in the terminal.

It also checks the type of NaN.

```
src > JS index.js > ...
1  import { divide } from './calculator.js';
2
3  let result = divide(2, 1);
4  console.log(result);
5  console.log(typeof(NaN));
```

5. Installing and Configuring Jest

Jest was used as the automated testing framework for this project.

The project was configured so that Jest could run the test file.

The Jest configuration is stored in:

jest.config.js

The project also contains the required Jest configuration in package.json.

```

() package.json > ...
1  {
2    "name": "automated-testing",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    > Debug
7    "scripts": {
8      "start": "node src/index.js",
9      "test": "node --experimental-vm-modules node_modules/jest/bin/jest.js"
10   },
11   "keywords": [],
12   "author": "",
13   "license": "ISC",
14   "type": "module",
15   "devDependencies": {
16     "jest": "^30.5.2"
17   }

```

```

JS jest.config.js > [⌕] default
1  export default {
2    |   testEnvironment: 'node',
3    |   transform: {}
4  };

```

6. Creating the Automated Tests

I created the test file:

__tests__/calculator.test.js

I created five tests in total.

There is one positive test and four negative tests.

Positive test

The positive test checks that the divide() function correctly divides two valid numbers.

Negative tests

The negative tests check invalid input:

1. A non-number first argument
2. A non-number second argument
3. NaN as an argument
4. Division by zero

The negative tests check that the function rejects invalid input and throws an error.

```

tests_ > JS calculator.test.js > describe('divide') callback
1  import { divide } from '../src/calculator.js';
2
3  describe('divide', () => {
4
5      it('divides two numbers correctly', () => {
6          expect(divide(10, 2)).toBe(5);
7      });
8
9      it('rejects a non-number first argument', () => {
10         expect(() => divide("10", 2))
11             .toThrow("Both arguments must be numbers");
12     });
13
14     it('rejects a non-number second argument', () => {
15         expect(() => divide(10, "2"))
16             .toThrow("Both arguments must be numbers");
17     });
18
19     it('rejects NaN as an argument', () => {
20         expect(() => divide(NaN, 2))
21             .toThrow("Arguments cannot be NaN");
22     });
23
24     it('rejects division by zero', () => {
25         expect(() => divide(10, 0))
26             .toThrow("Division by zero is not allowed");
27     });
28
29 });

```

7. Running the Automated Tests

After creating the tests, I ran the tests from the terminal using:

```
npm test
```

The tests were executed by Jest.

The result was:

Test Suites: 1 passed, 1 total

Tests: 5 passed, 5 total

All five tests passed successfully.

```
PS C:\Users\Käyttäj\OneDrive - University of Oulu and Oamk\Software testing\automated-testing> npm test

> automated-testing@1.0.0 test
> node --experimental-vm-modules node_modules/jest/bin/jest.js

(node:12932) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
PASS __tests__/calculator.test.js
  divide
    ✓ divides two numbers correctly (3 ms)
    ✓ rejects a non-number first argument (73 ms)
    ✓ rejects a non-number second argument (1 ms)
    ✓ rejects NaN as an argument
    ✓ rejects division by zero (1 ms)

Test Suites: 1 passed, 1 total
Tests:       5 passed, 5 total
Snapshots:   0 total
Time:        1.923 s
Ran all test suites.
```

8. Initializing Git

After completing the project, I used Git to keep track of the project files.

I initialized the Git repository with:

```
git init
```

I also created a .gitignore file.

The .gitignore file prevents unnecessary files such as node_modules from being added to the Git repository.

```
PS C:\Users\Käyttäj\OneDrive - University of Oulu and Oamk\Software testing\automated-testing> git init
Reinitialized existing Git repository in C:/Users/Käyttäj/OneDrive - University of Oulu and Oamk/Software tes
ting/automated-testing/.git/
PS C:\Users\Käyttäj\OneDrive - University of Oulu and Oamk\Software testing\automated-testing> █
```

9. Adding and Committing the Project

I added the required project files to Git and created a commit using `git commit -m "Add automated testing project"`. The commit was successfully created and contained 8 project files.

10. Creating the GitHub Repository

I created a GitHub repository for the project and connected my local project to it using `git remote add origin`. I then changed the branch to main and uploaded the project using `git push -u origin main`. The upload was successful, and the project is now available on GitHub.

11. Final GitHub Project

The completed project is available on GitHub:

<https://github.com/Mishomoon/automated-testing>

The repository contains the source code, automated tests, configuration files and test screenshot.

12. Final Result

The automated testing assignment was completed successfully.

The project contains:

- A Node.js application
- A divide() function
- Input validation
- Jest configuration
- One positive automated test
- Four negative automated tests
- A total of five automated tests
- A screenshot showing all tests passed
- Git version control
- A GitHub repository

The final test result was:

5 tests passed

1 test suite passed

The project was successfully uploaded to GitHub.